

```

In [ ]: # (0) IMPORT LIBRARIES

import numpy as np
import matplotlib.pyplot as plt
import scipy.signal as signal
from scipy.signal import butter, firwin, freqz, filtfilt, sosfiltfilt, sosfreqz
from scipy.io.wavfile import write as wav_write
import wfdb # Library for reading PhysioNet data
from scipy.signal import find_peaks


In [ ]: # (1) LOAD MIT-BIH REAL ECG SIGNAL

record_name = '100'
# Load the record. We take 100 seconds (360 samples/sec * 100 sec = 36000 samples)
record = wfdb.rdrecord(record_name, sampsto=36000, pn_dir='mitdb/')

# Extract the sampling frequency
fs = record.fs # fs = 360 Hz
duration = len(record.p_signal) / fs
t = np.arange(0, duration, 1/fs)
N = len(t)
# Extract Lead II signal (Column 0) as the clean reference
ecg_noisy = record.p_signal[:, 0]
print(f"Loaded MIT-BIH Record: {record_name}. Fs: {fs} Hz. Length: {duration:.2f} s")

plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy)
plt.title("Raw MIT-BIH Signal (Magnitude)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(0, 5)
plt.grid(True)
plt.show()


In [ ]: # (2) FFT FUNCTION AND original signal PLOT IN FREQUENCY DOMAIN

# Set the signal to be analyzed (the raw input signal from MIT-BIH)
signal_to_analyze = ecg_noisy
N = len(signal_to_analyze)

# Calculate the FFT (using rfft for real signal efficiency)
# fft_signal contains the magnitude
fft_signal = np.abs(np.fft.rfft(signal_to_analyze))
# Calculate the corresponding frequencies
freq = np.fft.rfftfreq(N, 1/fs)

# Drawing FFT plot
plt.figure(figsize=(10, 3))

# Plotting the positive half of the spectrum
# Note: rfft already gives you only the positive half.
plt.plot(freq, fft_signal, label="Raw ECG Input Spectrum", alpha=0.8)

# Setting Limits and Labels
plt.xlim(0, fs/2) # Display up to the Nyquist frequency
plt.ylim(0, np.max(fft_signal[1:]) * 1.1) # Adjusted Y-limit (Magnitude)
plt.title(f"FFT of Raw MIT-BIH Signal (Magnitude)")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.legend()
plt.show()


In [ ]: # (3) DESIGN FILTERS (Cascade HPF/LPF)

# 3.1 HPF (High-Pass Filter) - To remove Baseline Wander (< 0.5 Hz)
hpf_lowcut = 0.5
# IIR HPF (Butterworth)
iir_hpf_order = 4
sos_iir_hpf = signal.butter(iir_hpf_order, hpf_lowcut, btype='highpass', fs=fs, output='sos')

# FIR HPF (Hamming)
fir_hpf_order = 101
b_fir_hpf = firwin(fir_hpf_order, hpf_lowcut, pass_zero='highpass', fs=fs, window='hamming')
a_fir_hpf = [1]

# 3.2 LPF (Low-Pass Filter) - For removing muscle tremor (> 40 Hz)
lpf_highcut = 40.0
# IIR LPF (Butterworth)
iir_lpf_order = 4
sos_iir_lpf = signal.butter(iir_lpf_order, lpf_highcut, btype='lowpass', fs=fs, output='sos')
# FIR LPF (Hamming)
fir_lpf_order = 101
b_fir_lpf = firwin(fir_lpf_order, lpf_highcut, pass_zero='lowpass', fs=fs, window='hamming')
a_fir_lpf = [1]

```

```
In [ ]: # (4) FILTER ECG SIGNALS (Cascade Implementation - Zero Phase)
```

```
# --- 4.1 IIR Cascade ---
# 1. HPF IIR
ecg_hpf_iir = sosfiltfilt(sos_iir_hpf, ecg_noisy)
# 2. LPF IIR
ecg_iir_filtered = sosfiltfilt(sos_iir_lpf, ecg_hpf_iir)

# --- 4.2 FIR Cascade ---
# 1. HPF FIR
ecg_hpf_fir = filtfilt(b_fir_hpf, a_fir_hpf, ecg_noisy)
# 2. LPF FIR
ecg_fir_filtered = filtfilt(b_fir_lpf, a_fir_lpf, ecg_hpf_fir)
```

```
In [ ]: # (5) Calculating SNR before and after filtration
```

```
def calc_snr_db(reference_signal, processed_signal):
    # Noise is the difference between the clean reference and the processed signal
    noise_est = processed_signal - reference_signal
    P_signal = np.mean(reference_signal**2)
    P_noise = np.mean(noise_est**2)

    if P_noise < 1e-15: return 999.0
    snr_db = 10 * np.log10(P_signal / P_noise)
    return snr_db

# NOTE: In a real scenario, the ECG input is already noisy. For evaluation,
# we use the raw signal as the 'reference' to compare how much the filtered
# signal deviates from the original, a common metric in filtering performance.

snr_after_fir = calc_snr_db(ecg_noisy, ecg_fir_filtered)
snr_after_iir = calc_snr_db(ecg_noisy, ecg_iir_filtered)

print("===== FINAL CASCADE SNR Comparison =====")
print(f"SNR after FIR Cascade           : {snr_after_fir:.2f} dB")
print(f"SNR after IIR Cascade             : {snr_after_iir:.2f} dB")
```

```
In [ ]: # (6) FREQUENCY RESPONSE (Magnitude) - CASCADE
```

```
worN_points = 4096

# IIR Components (HPF & LPF) - Using sosfreqz
w_ihpf, h_ihpf = sosfreqz(sos_iir_hpf, worN=worN_points, fs=fs)
w_ilpf, h_ilpf = sosfreqz(sos_iir_lpf, worN=worN_points, fs=fs)

# IIR Total Cascade Response
h_iir_cascade = h_ihpf * h_ilpf
w_iir_cascade = w_ihpf

# FIR Components
w_fhpf, h_fhpf = freqz(b_fir_hpf, a_fir_hpf, worN=worN_points, fs=fs)
w_flpf, h_flpf = freqz(b_fir_lpf, a_fir_lpf, worN=worN_points, fs=fs)

# FIR Total Cascade Response
h_fir_cascade = h_fhpf * h_flpf
w_fir_cascade = w_fhpf

plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, 20*np.log10(np.abs(h_fir_cascade)+1e-12), label="FIR Cascade", color='green')
plt.plot(w_iir_cascade, 20*np.log10(np.abs(h_iir_cascade)+1e-12), label="IIR Cascade", color='red', linestyle='--')
plt.title("Cascade Magnitude Response (dB) - FIR vs IIR Efficiency")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Magnitude [dB]")
plt.grid(True)
plt.legend()
plt.show()
```

```
In [ ]: # (6.1) Frequency Response (Linear magnitude) - Cascade
```

```
plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, np.abs(h_fir_cascade), label="FIR Cascade Magnitude", color='green')
plt.plot(w_iir_cascade, np.abs(h_iir_cascade), label="IIR Cascade Magnitude", color='red', linestyle='--')
plt.title("Frequency Response of Filter Cascade (Linear Magnitude)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()
plt.show()
```

```
In [ ]: # (7) Phase Response
```

```
phase_fir_cascade = np.unwrap(np.angle(h_fir_cascade))
phase_iir_cascade = np.unwrap(np.angle(h_iir_cascade))

plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, phase_fir_cascade, label="FIR Cascade (Linear)", color='green')
plt.plot(w_iir_cascade, phase_iir_cascade, label="IIR Cascade (Non-Linear)", color='red', linestyle='--')
plt.title("Cascade Phase Response")
```

```
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.grid(True)
plt.legend()
plt.show()
```

In []: # (7.1) Phase Response zoomed

```
plt.figure(figsize=(10,4))
plt.plot(w_fir_cascade, phase_fir_cascade, color='green')
plt.title("FIR Cascade Phase Response zoomed(Linear Phase)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.xlim(0, 200)
plt.ylim(-300, 2)
plt.grid(True)
plt.show()

plt.figure(figsize=(10,4))
plt.plot(w_iir_cascade, phase_iir_cascade, color='red', linestyle='--')
plt.title("IIR Cascade Phase Response zoomed (Non-Linear Phase)")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Phase [rad]")
plt.xlim(0, 75)
plt.ylim(-15, 2)
plt.grid(True)
plt.show()
```

In []: # (8) GROUP DELAY - CASCADE

```
worN_points = 4096
w_freq = np.linspace(1.0, fs / 2 - 0.1, worN_points)

# Reconstructing IIR transactions in b, a format for Group Delay (GD) analysis
# (We need this because group_delay does not accept the SOS format directly)
b_iir_hpf, a_iir_hpf = signal.butter(iir_hpf_order, hpf_lowcut, btype='highpass', fs=fs, output='ba')
b_iir_lpf, a_iir_lpf = signal.butter(iir_lpf_order, lpf_highcut, btype='lowpass', fs=fs, output='ba')

# Calculate Group Delay for each component
# 1. IIR HPF Group Delay

w_ihpf_gd, gd_ihpf = signal.group_delay((b_iir_hpf, a_iir_hpf), w_freq, fs=fs)

# 2. IIR LPF Group Delay

w_ilpf_gd, gd_ilpf = signal.group_delay((b_iir_lpf, a_iir_lpf), w_freq, fs=fs)

# Total Group Delay of the Sequence (Cascade)
gd_iir_cascade = gd_ihpf + gd_ilpf

# --- FIR Group Delay ---
delay_fir_total = (fir_hpf_order - 1) // 2 + (fir_lpf_order - 1) // 2
gd_fir_cascade = np.full(len(w_freq), delay_fir_total)
w_gd_cascade = w_freq

plt.figure(figsize=(10,4))
plt.plot(w_gd_cascade, gd_fir_cascade, label="FIR Cascade (Constant)", color='green')
plt.plot(w_gd_cascade, gd_iir_cascade, label="IIR Cascade (Variable)", color='red', linestyle='--')
plt.title("Cascade Group Delay")
plt.xlabel("Frequency [Hz]")
plt.ylabel("Delay [samples]")
plt.ylim(0, np.max(gd_fir_cascade) * 3)
plt.grid(True)
plt.legend()
plt.show()
```

In []: # (9) Time-domain Comparison (Full View)

```
plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy, label="Raw ECG Input", color='blue')
plt.plot(t, ecg_fir_filtered, label="FIR Cascade Filtered", color='green')
plt.plot(t, ecg_iir_filtered, label="IIR Cascade Filtered", color='red')
plt.title("Time-domain Comparison (Full Signal)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(0, 5)
plt.grid(True)
plt.legend()
plt.show()
```

In []: # (9.1) Zoomed Version Time Domain - To display P, QRS, T and distortion

```
plt.figure(figsize=(10,3))
plt.plot(t, ecg_noisy, label="Raw ECG Input", color='blue')
plt.plot(t, ecg_fir_filtered, label="FIR Cascade Filtered", color='green')
plt.plot(t, ecg_iir_filtered, label="IIR Cascade Filtered", color='red')
plt.title("Time-domain Comparison (Zoomed on P-QRS-T)")
```

```
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 2)
plt.ylim(-1.5, 2)
plt.grid(True)
plt.legend()
plt.show()
```

In []: # (10) FFT CALCULATIONS (Magnitude)

```
# 1. Raw ECG Input (ecg_noisy)
freq_noisy = np.fft.rfftfreq(N, 1/fs)
fft_noisy = np.abs(np.fft.rfft(ecg_noisy))

# 2. FIR Filtered Signal
freq_fir = np.fft.rfftfreq(N, 1/fs)
fft_fir = np.abs(np.fft.rfft(ecg_fir_filtered))

# 3. IIR Filtered Signal
freq_iir = np.fft.rfftfreq(N, 1/fs)
fft_iir = np.abs(np.fft.rfft(ecg_iir_filtered))

# FFT Magnitude Comparison (Filtered vs Raw)

plt.figure(figsize=(10, 4))

plt.plot(freq_noisy, fft_noisy, label="Raw ECG Input", color='blue', alpha=0.5)
plt.plot(freq_fir, fft_fir, label="FIR Cascade Filtered", color='green')
plt.plot(freq_iir, fft_iir, label="IIR Cascade Filtered", color='red', linestyle='--')

plt.xlim(0, fs/2)
plt.ylim(0, np.max(fft_noisy[1:]) * 1.1)
plt.title("FFT Magnitude Comparison - Filter Performance")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid(True)
plt.legend()
plt.show()
```

In []: # (11) SAVE SIGNALS

```
wav_write("D:/project/ecg_noisy.wav", int(fs), (ecg_noisy/np.max(np.abs(ecg_noisy))*32767).astype(np.int16))
wav_write("D:/project/ecg_fir_filtered.wav", int(fs), (ecg_fir_filtered/np.max(np.abs(ecg_fir_filtered))*32767).astype(np.int16))
wav_write("D:/project/ecg_iir_filtered.wav", int(fs), (ecg_iir_filtered/np.max(np.abs(ecg_iir_filtered))*32767).astype(np.int16))

print("All signals saved as WAV for listening and further analysis.")
```

In []: # (12) Calculate R-Peak, HR, HRV, and Arrhythmia

```
def analyze_ecg_performance(processed_signal, fs, max_peak_height_ratio=0.5, min_distance_samples=36):
    """
    Calculates R-Peaks, RR Intervals, HR, HRV (SDNN, RMSSD) and Arrhythmia detection.
    """

    # 1. R-Peak Detection
    # 36 samples = 0.1 seconds at fs=360 Hz.
    r_peaks_indices, _ = find_peaks(
        processed_signal,
        height=(np.max(processed_signal) * max_peak_height_ratio),
        distance=min_distance_samples
    )

    # 2. Calculate R-R Intervals (in seconds and milliseconds)
    rr_intervals_samples = np.diff(r_peaks_indices)
    rr_intervals_seconds = rr_intervals_samples / fs
    rr_intervals_ms = rr_intervals_seconds * 1000 # Convert to milliseconds

    # 3. Calculate Heart Rate (HR)
    hr_bpm_instantaneous = 60 / rr_intervals_seconds
    average_hr = np.mean(hr_bpm_instantaneous)

    # 4. HRV Analysis (SDNN, RMSSD)
    sdnn = np.std(rr_intervals_ms) # Standard Deviation of NN Intervals (SDNN)
    diff_rr = np.diff(rr_intervals_ms)
    rmssd = np.sqrt(np.mean(diff_rr**2)) # Root Mean Square of Successive Differences (RMSSD)

    # 5. Simple Arrhythmia Detection (25% deviation from the mean)
    # A beat is considered abnormal if its RR interval deviates by 25% from the mean
    avg_rr_ms = np.mean(rr_intervals_ms)
    abnormal_threshold_lower = avg_rr_ms * 0.75
    abnormal_threshold_upper = avg_rr_ms * 1.25

    abnormal_beats_count = 0
    abnormal_rr_indices = []
    total_beats = len(rr_intervals_ms) + 1

    for i, rr in enumerate(rr_intervals_ms):
        if (rr < abnormal_threshold_lower) or (rr > abnormal_threshold_upper):
```

```

        abnormal_beats_count += 1
        # Store the index of the beat that followed the abnormal interval
        abnormal_rr_indices.append(r_peaks_indices[i + 1])

    abnormal_percentage = (abnormal_beats_count / total_beats) * 100

    return {
        'r_peaks_indices': r_peaks_indices,
        'rr_intervals_seconds': rr_intervals_seconds,
        'rr_intervals_ms': rr_intervals_ms,
        'average_hr': average_hr,
        'sdnn': sdnn,
        'rmssd': rmssd,
        'abnormal_beats_count': abnormal_beats_count,
        'abnormal_percentage': abnormal_percentage,
        'abnormal_rr_indices': abnormal_rr_indices,
        'total_beats': total_beats
    }

```

In []: # (13) Apply analysis to FIR and IIR signals

```

# FIR signal analysis
fir_analysis = analyze_ecg_performance(ecg_fir_filtered, fs)
r_peaks_indices_fir = fir_analysis['r_peaks_indices']
rr_intervals_seconds_fir = fir_analysis['rr_intervals_seconds']
average_hr_fir = fir_analysis['average_hr']

# IIR signal analysis
iir_analysis = analyze_ecg_performance(ecg_iir_filtered, fs)
r_peaks_indices_iir = iir_analysis['r_peaks_indices']
rr_intervals_seconds_iir = iir_analysis['rr_intervals_seconds']
average_hr_iir = iir_analysis['average_hr']

processed_signal = ecg_fir_filtered
r_peaks_indices = r_peaks_indices_fir
abnormal_beats_count = fir_analysis['abnormal_beats_count']
abnormal_rr_indices = fir_analysis['abnormal_rr_indices']
average_hr = average_hr_fir

# Print Results

print("=" * 30)
print("--- FIR FILTERED SIGNAL ANALYSIS ---")
print(f"Total R-Peaks Detected: {len(r_peaks_indices_fir)}")
print(f"Average Heart Rate (BPM): {average_hr_fir:.2f} BPM")
print("-" * 30)
print("HRV & Arrhythmia (FIR):")
print(f"SDNN: {fir_analysis['sdnn']:.2f} ms | RMSSD: {fir_analysis['rmssd']:.2f} ms")
print(f"Abnormal Beats Detected: {fir_analysis['abnormal_beats_count']} ({fir_analysis['abnormal_percentage']:.2f}%)")

print("\n" + "=" * 30)
print("--- IIR FILTERED SIGNAL ANALYSIS ---")
print(f"Total R-Peaks Detected: {len(r_peaks_indices_iir)}")
print(f"Average Heart Rate (BPM): {average_hr_iir:.2f} BPM")
print("-" * 30)
print("HRV & Arrhythmia (IIR):")
print(f"SDNN: {iir_analysis['sdnn']:.2f} ms | RMSSD: {iir_analysis['rmssd']:.2f} ms")
print(f"Abnormal Beats Detected: {iir_analysis['abnormal_beats_count']} ({iir_analysis['abnormal_percentage']:.2f}%)")
print("=" * 30)

```

In []: #(14)

```

# Setup FIR plotting variables
abnormal_t_fir = t[fir_analysis['abnormal_rr_indices']]
abnormal_amp_fir = ecg_fir_filtered[fir_analysis['abnormal_rr_indices']]

# Setup IIR plotting variables
abnormal_t_iir = t[iir_analysis['abnormal_rr_indices']]
abnormal_amp_iir = ecg_iir_filtered[iir_analysis['abnormal_rr_indices']]

plt.figure(figsize=(12, 8))

# Top plot for FIR
plt.subplot(2, 1, 1)
plt.plot(t, ecg_fir_filtered, label="FIR Filtered ECG (Linear Phase)", color='green', alpha=0.8)
plt.plot(t[r_peaks_indices_fir], ecg_fir_filtered[r_peaks_indices_fir], 'go', markersize=5, label="FIR R-Peaks Detected")

# Plotting Abnormal R-Peaks (FIR)
if fir_analysis['abnormal_beats_count'] > 0:
    plt.plot(abnormal_t_fir, abnormal_amp_fir, 'x', markersize=8, color='orange', markeredgecolor='black', label="Abnormal R-Peak (FIR)")

plt.title(f"TOP: FIR Filter (Accurate HR: {average_hr_fir:.2f} BPM, Abnormal: {fir_analysis['abnormal_percentage']:.2f}%)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 5) # Zoom on a small section
plt.grid(True)
plt.legend(loc='lower right')

```

```
# Bottom plot for IIR
plt.subplot(2, 1, 2)
plt.plot(t, ecg_iir_filtered, label="IIR Filtered ECG (Phase Distorted)", color='red', alpha=0.8)
plt.plot(t[r_peaks_indices_iir], ecg_iir_filtered[r_peaks_indices_iir], 'bo', markersize=5, label="IIR R-Peaks Detected")

# Plotting Abnormal R-Peaks (IIR)
if iir_analysis['abnormal_beats_count'] > 0:
    plt.plot(abnormal_t_iir, abnormal_amp_iir, 'x', markersize=8, color='orange', markeredgecolor='black', label="Abnormal R-Peak (IIR)")

plt.title(f"BOTTOM: IIR Filter (HR: {average_hr_iir:.2f} BPM, Abnormal: {iir_analysis['abnormal_percentage']:.2f}%)")
plt.xlabel("Time [s]")
plt.ylabel("Amplitude")
plt.xlim(1, 5) # Zoom on a small section
plt.grid(True)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

In []:

In []: